Asymptotic Analysis of Machine Learning Training Algorithms: How Big-O Applies to Model Training



By

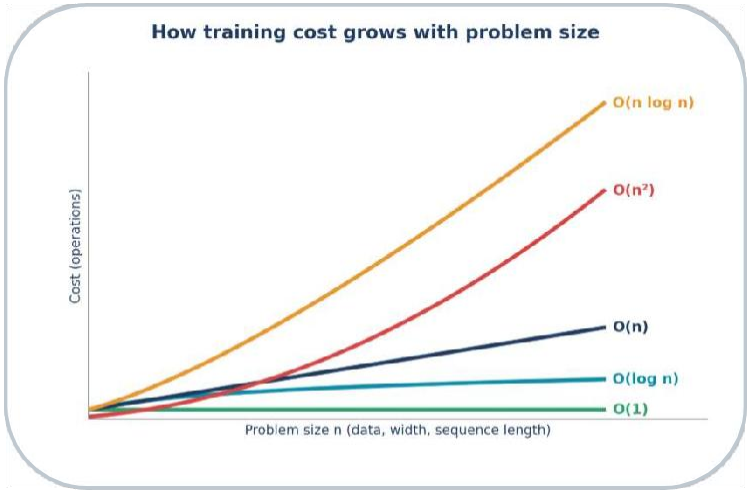
Name: Kataru Tharun Kumar(92400117129)

Table of Contents.

Sl. No.	Contents Details	Slide Number(SLN)
1	Introduction	SLN1
2	Review of Related Work	SLN2
3	Research Gap	SLN3
4	Proposed Diagram	SLN4
5	Flowchart	SLN5
6	Future Plan and Conclusion	SLN6–7
7	References	SLN8

INTRODUCTION

- **Big-O (asymptotic) analysis** describes how cost grows with problem size. In model training, total cost is the cost of one step multiplied by the number of steps needed to reach a target accuracy.
- **Bach and Moulines [1]:** for SGD with step size $\gamma_n = C \cdot n^{-\alpha}$, the error falls as O(1/n) when the problem is strongly convex. Averaging with α in (1/2, 1) stays near-optimal in all cases.
- **Katharopoulos et al. [2]:** linear attention reduces self-attention from $O(N^2)$ to O(N) in time and memory, and makes autoregressive inference cost constant per step.
- **Sirignano and Spiliopoulos [3]:** as hidden units $N \to \infty$ and steps grow as $k = N \cdot t$, a Q-network converges to a limit ODE whose fixed point is the Bellman solution.
- **This study** combines the three views to show how Big-O applies to model training.



REVIEW RELATED WORK

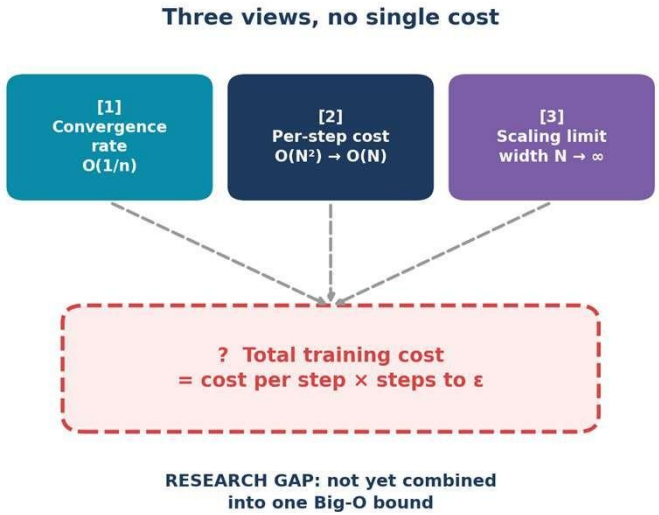
Table 1: Summary of existing asymptotic analyses of machine learning training algorithms.

Related Research	Technology	Environment	Methodology/ Algorithm	Advantage	**Limitation**
Bach and Moulines [1]	Stochastic approximation, convex optimization	Convex losses on a Hilbert space (least-squares, logistic)	SGD and Polyak-Ruppert averaging, $\gamma_n = C \cdot n^{-\alpha}$	Non-asymptotic bounds; averaging with α in (1/2, 1) is near-optimal	α = 1 is not robust to the constant C or lack of strong convexity
Katharopoulos et al. [2]	Deep learning, Transformers	Image generation (MNIST, CIFAR-10), speech recognition (WSJ)	Linear attention with feature map φ(x) = elu(x) + 1; RNN form	O(N) time and memory instead of $O(N^2)$; constant cost per inference step	Higher error than softmax on WSJ (PER 8.08 vs 5.12)
Sirignano and Spiliopoulos [3]	Reinforcement learning, mean-field analysis	Single-layer Q-network, finite MDP, width $N \to \infty$	Q-learning and SGD with $1/\sqrt{N}$ scaling, rate α/N, k = N·t steps	Limit ODE has a unique fixed point: the Bellman solution	Convergence proved only for $\gamma < 2/(1+K)$; one hidden layer only

RESEARCH GAP

The following research gaps motivate this study.

- **No unified cost:** [1] gives convergence rates, [2] gives per-step cost and [3] gives scaling limits, but none combines them into one total training cost.
- **Constants matter:** bounds in [1] depend on Lipschitz constants and on C, and can be locally pessimistic. An adaptive learning-rate sequence is still open.
- **Speed vs accuracy:** linear attention in [2] is much faster, but softmax still gives lower phoneme error on WSJ (5.12 vs 8.08).
- **Limited scope:** [3] proves convergence only for $\gamma < 2/(1+K)$ and for single-layer networks; multilayer networks are open.



Base paper: Bach and Moulines [1], chosen for its explicit non-asymptotic bounds on training convergence.

PROPOSED DIAGRAM

Overview of the proposed three-layer framework for applying Big-O analysis to model training.

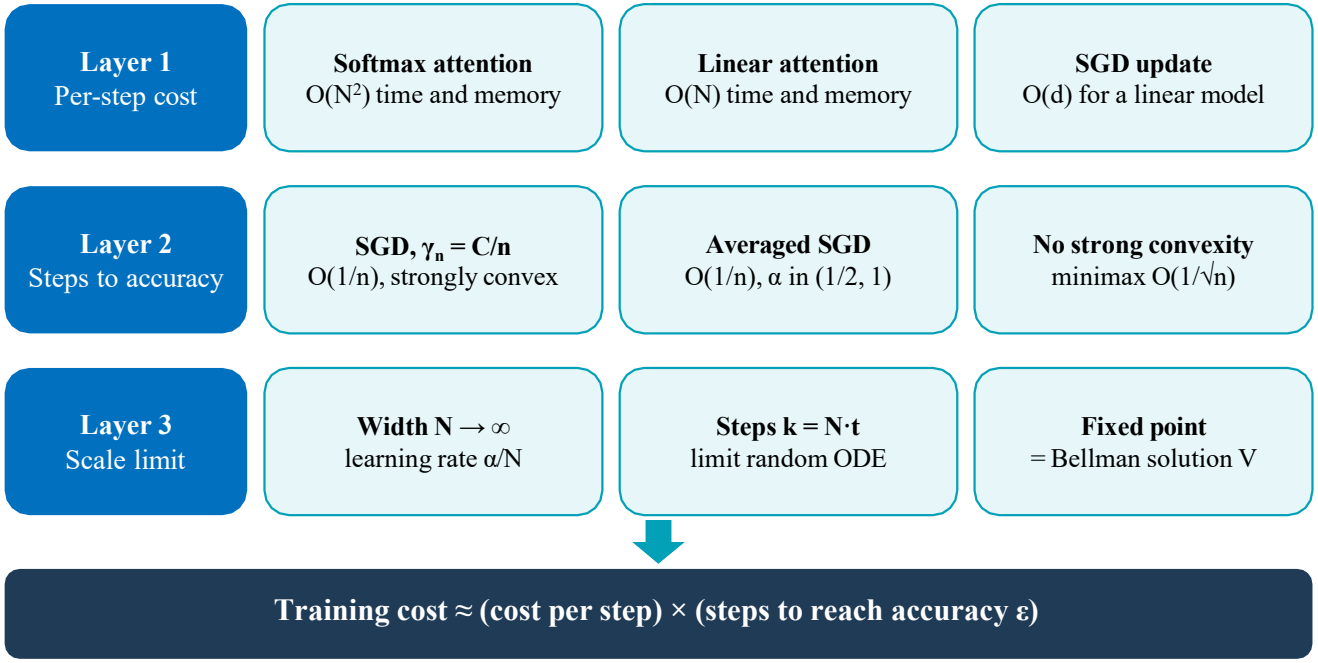


Figure 1: Three-layer framework for Big-O analysis of machine learning training cost

Layer 1, per-step cost – counts operations per update in terms of N, d, C, and M [2].

Layer 2, convergence – bounds the steps needed, using step size $\gamma_n = C \cdot n^{-\alpha}$ [1].

Layer 3, scale limit – studies width $N \to \infty$ and steps $k = N \cdot t$ to obtain a limit ODE [3].

METHODOLOGICAL FLOW DIAGRAM

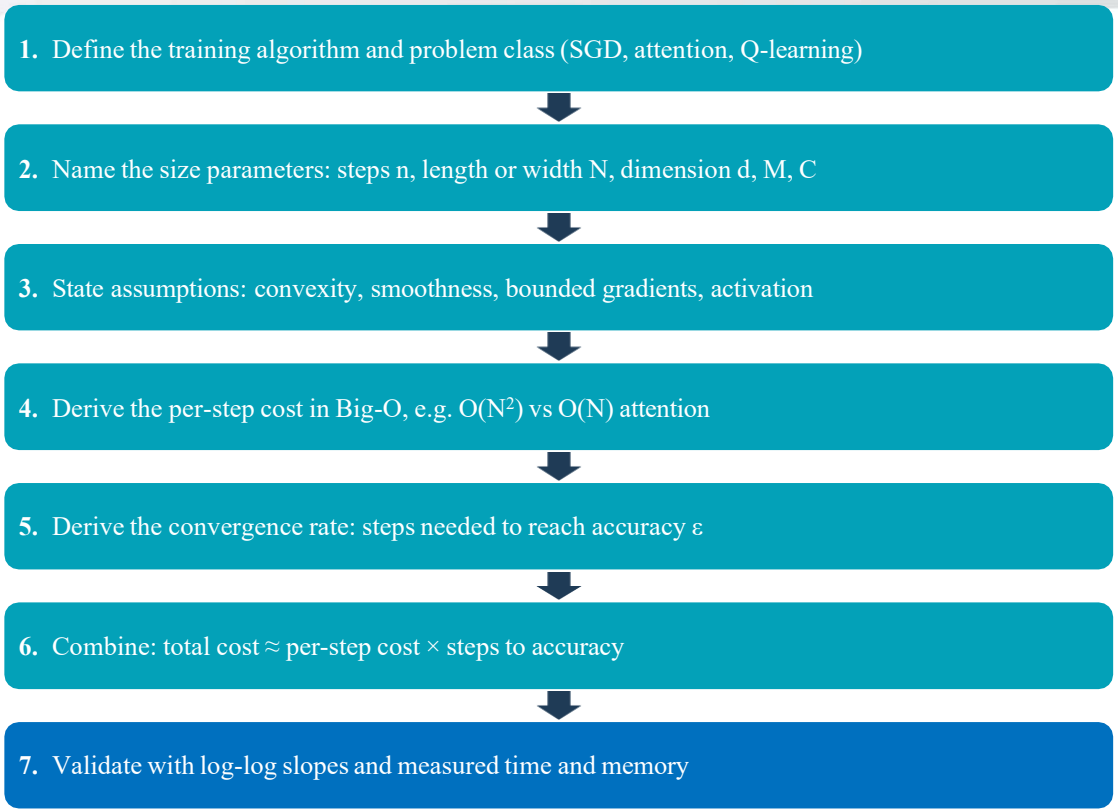


Figure 2: Flow of the Big-O analysis of a machine learning training algorithm.

- **Steps 1–3** fix the object of study: the algorithm, the quantities that grow, and the assumptions behind a bound. In [1], results change when strong convexity is assumed or dropped.
- **Steps 4–5** give the two Big-O factors. In [2], linear attention costs O(NCM) instead of $O(N^2 \max(D, M))$. In [1], the rate depends on the exponent α of $\gamma_n = C \cdot n^{-\alpha}$.
- **Steps 6–7** multiply the factors and check them by experiment. Inference in [2] was up to 4000× faster, and simulations in [1] agree with the predicted rates.

FUTURE PLAN

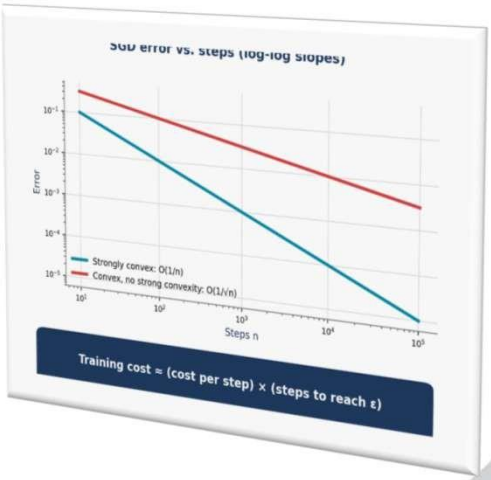
- **Total training-cost analysis**
  Combine per-step cost and convergence rate into one Big-O bound for SGD, linear attention and Q-learning.
- **Adaptive learning rates**
  Design a truly adaptive step-size sequence, since the fixed-constant bounds in [1] can be locally pessimistic.
- **Deeper networks**
  Extend the limit-ODE analysis in [3] to multilayer networks and to all discount factors γ in (0, 1).
- **Better feature maps for linear attention**
  Test kernel approximations such as random Fourier features, which could allow reuse of softmax-pretrained models [2].
- **Experimental validation**
  Measure log-log convergence slopes and GPU time and memory to check the predicted Big-O behaviour.

CONCLUSION

From this study, I conclude that Big-O applies to model training in two parts: the cost of each step and the number of steps needed to reach a target accuracy.

Cost per step: linear attention reduces self-attention from $O(N^2)$ to $O(N)$ and keeps inference cost constant per step [2].

- **Number of steps:** SGD with $\gamma_n = C/n$ reaches $O(1/n)$ on strongly convex problems but is not robust to the constant C. Averaging with a slower decay gives near-optimal rates in all cases [1].
- **Scale:** as width $N \to \infty$ with learning rate α/N, a single-layer network follows a limit ODE. For Q-learning its fixed point is the Bellman solution, and for regression it reaches zero training loss [3].
- **Together:** training cost ≈ (cost per step) × (steps to reach accuracy ε).
- **Limitation:** this review rests on three papers and their assumptions (convexity, smoothness, $\gamma < 2/(1+K)$); no new experiments were run.



REFERENCES

[1] Bach, F., & Moulines, E. (2011). Non-asymptotic analysis of stochastic approximation algorithms for machine learning. *Advances in Neural Information Processing Systems (NIPS)*, 24.

[2] Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F. (2020). Transformers are RNNs: Fast autoregressive transformers with linear attention. *Proceedings of the 37th International Conference on Machine Learning*, PMLR 119.

[3] Sirignano, J., & Spiliopoulos, K. (2022). Asymptotics of reinforcement learning with neural networks. *Stochastic Systems*, 12(1), 2–29.

THANK YOU

